Repository: https://github.com/pulkittiwari-in/SmartDispatch

SmartDispatch — Zomato-Inspired Delivery Partner Assignment & ETA Engine

Individual 24-hour challenge | Python 3.10+ | Synthetic data only

Scenario

A food-delivery service assigns orders to delivery partners. Partner availability, existing workload, restaurant preparation time and travel estimates affect the delivery ETA. Build the decision logic; do not build a mobile app, map UI or live GPS integration.

This is a simulated challenge inspired by food-delivery dispatch and ETA problems associated with services such as Zomato. It is not Zomato's internal software, is not affiliated with Zomato, and uses only synthetic data.

Required behaviour

1. Validate order ID, restaurant ID, destination zone and non-negative preparation/travel times.

2. Consider only active partners below their order capacity. A candidate cannot receive a reliable ETA if any required travel-time entry is missing.

3. Calculate pickup time as current time + partner-to-restaurant travel + remaining workload. Calculate ETA as pickup time + restaurant preparation + restaurant-to-customer travel.

4. Pick the eligible candidate with the lowest ETA. Break ETA ties by partner ID in ascending lexicographic order.

5. Missing travel data must never be treated as zero. If no valid candidate remains, leave the order unassigned and explain why.

6. Process a cancellation event only once so a duplicate cancellation does not release workload/capacity twice.

7. Return deterministic results and handle empty lists, unknown IDs, malformed records and negative time values safely.

Setup

Run: `python -m pip install -r requirements-dev.txt`, then `python -m pytest -q`, then `python -m main`.

Repository reading checklist and dataset

Before editing, read `README.md` and `STUDENT_START_HERE.md`, then inspect the whole repository—not only this PDF. Read `CONTRIBUTING.md`, `.github/PULL_REQUEST_TEMPLATE.md`, the test/template documents, `service.py`, `models.py`, `fixtures.py`, `main.py`, every file under `tests/`, and the workflow files under `.github/workflows/`.

The expanded synthetic dataset is in `data/records.json`: **180 orders, 60 delivery-partner records, 387 travel-time records, and 177 cancellation events**, plus an edge-case catalog. Read `data/README.txt` for its schema. Some travel routes are deliberately missing; the dataset also covers full capacity, inactive partners, ETA boundary values, tie-breaking, duplicate cancellations and invalid inputs. Convert the JSON travel-time rows into the tuple-key mapping expected by `service.py`. The offline demo uses `fixtures.py`; the large JSON is additional investigation and test material.

Run the baseline tests before editing and record their actual result. Do not delete, skip, weaken or bypass tests.

Work to submit

- Read the source and run the starter tests before editing.
- Treat the required behaviour above as the acceptance contract. Investigate failing assertions, locate the root cause and fix the logic instead of hard-coding fixture results.
- Add regression tests and document at least five edge cases in `TESTING.md`.
- Submit one pull request to the original repository containing your code and tests. Do not merge it.
- Include `DISPATCH_NOTES.md` with one worked ETA, one no-eligible-partner example and three edge cases.

Mandatory problem-solving documentation — all three files required

Submit these three completed Markdown files at the repository root in your pull request. They are evaluated; code alone is not a complete submission.

- `approach.md` — record your starting test result, the code/data flow you explored, key hypotheses, experiments you ran, implementation strategy, and why you chose it.
- `challenges.md` — **COMPULSORY**. Describe concrete obstacles or confusing behaviours, what you tried, evidence from tests/logs, how you resolved them, and any unresolved limitations. If you did not get stuck, state that honestly and describe the hardest edge case you deliberately tested; do not invent a struggle.
- `optimizations.md` — describe optimizations considered or implemented, why you chose or rejected them, correctness/readability/performance trade-offs, and future improvements. Separate measured improvements from ideas you did not measure.

Keep the notes concise, specific and evidence-based. Describe your actual work, tests and technical decisions; do not provide private hidden chain-of-thought or a stream-of-consciousness transcript. Generic or copied text does not demonstrate your process. Each file is worth 5 marks. A missing or empty file receives 0/5 for that component and makes the documentation requirement incomplete.

Optional stretch work

Choose at most one after the core works: batch-assign orders in supplied priority order; return an explanation for each candidate; or recalculate ETA after a preparation-time change.

Testing and GitHub Actions

Some visible tests are expected to fail on the untouched starter due to incomplete behaviour. The exact test count is not fixed. GitHub Actions runs on pushes and pull-request updates and reports visible test counts. A red check before fixes is expected. Do not remove, skip, weaken or bypass tests or edit CI to evade failures.

Final submission checklist — complete every item

Before the deadline, verify each item below. **A code change alone is not a complete submission.**

- [ ] I read this problem statement, `STUDENT_START_HERE.md`, `README.md`, the contribution guide, all relevant source files, the synthetic data documentation, every visible test, and the GitHub Actions workflow.
- [ ] I ran `python -m pip install -r requirements-dev.txt`, recorded the untouched baseline result from `python -m pytest -q`, and ran `python -m main`.
- [ ] I fixed the underlying causes of relevant failures and added regression tests. I did not delete, skip, weaken, hard-code around, or bypass existing tests or edit CI to hide failures.
- [ ] I completed the existing `approach.md`, `challenges.md`, and `optimizations.md` templates at the repository root. Each must describe my own work and evidence; `challenges.md` is compulsory. Each is worth 5 marks.
- [ ] I completed `TESTING.md` with the real Python version, commands and test summary, plus at least five edge cases showing input, expected behaviour, actual result, and test/evidence.
- [ ] I completed `DISPATCH_NOTES.md`. Include (1) one worked ETA calculation, (2) one example where no partner is eligible, (3) assumptions, and (4) at least three edge cases.
- [ ] I reviewed the diff, kept the change focused, and ensured I did not commit secrets, credentials, or unrelated files.
- [ ] I opened **one pull request from my fork to this original repository**, included code, tests and all required documents, filled in the pull-request template, and **did not merge the pull request**.
- [ ] I checked the pull-request CI result and documented any remaining failures honestly.
- [ ] I can explain the changes independently and make a small live modification if requested.

If any item is incomplete, finish it before submitting. Record actual results only; never claim a test passed unless it was run. Follow the individual-work and AI-use policy below.

Mandatory problem-solving files — submit all three

You must complete **all three files** below in the root of your assigned repository and include them in your pull request. These are required deliverables and together are worth **15 marks** in the evaluation rubric (5 marks each). Start from the provided repository templates, replace the placeholder prompts with your own work, and commit the completed files. A blank template or generic, unsupported text does not count as completion.

1. `approach.md` — approach and implementation decisions (5 marks)

Explain your engineering approach in your own words:

- How you understood the problem and traced the relevant code/data flow.
- The root cause you identified and the evidence that led you to it.
- The solution options you considered and why you selected your approach.
- Important implementation decisions, experiments, and how you validated the fix.

2. `challenges.md` — challenges and debugging (5 marks; compulsory)

Record the real challenges you faced while investigating and solving the task:

- The specific technical issue or obstacle.
- What you tried, what worked or failed, and how you investigated it.
- Evidence such as reproduction steps, relevant error/test output, or code references.
- How you resolved the issue and any remaining limitations.

Do not invent problems or results. If you did not encounter a major obstacle, explain a genuine investigation or tricky edge case you worked through and support it with evidence.

3. `optimizations.md` — optimizations and trade-offs (5 marks)

Describe improvements and engineering trade-offs:

- Which optimizations you considered and which you actually implemented.
- Why an optimization is useful, with test results or other evidence where possible.
- Trade-offs, risks, complexity, performance, or maintainability considerations.
- What you would improve next and why, if more time were available.

Documentation submission rules

- Use your own words and document work you actually performed; do not copy another student's work.
- Be concrete and evidence-based. Reference the tests you ran and report their actual outcomes—never claim a test passed unless you ran it.
- Include all three completed files in the same pull request as your code and tests.
- **Missing, empty, or materially incomplete files can receive zero for the relevant 5-mark component and may make the submission incomplete.**
- You must be able to explain your decisions and evidence during the evaluation. This asks for an engineering record, not a private chain-of-thought transcript.

Individual-work policy

AI chatbots, AI coding assistants, generated patches/tests and another person's solution-writing/debugging are prohibited. Ordinary non-AI documentation is allowed. Do not paste challenge code/data into external services. You must explain your work and make a small live change if asked. Code walkthroughs can support integrity checks, but no automated detector proves AI use.

Rubric — 100 marks

- Codebase understanding and ETA trace: 10
- Assignment logic and deterministic tie-breaking: 20
- Edge-case safety, capacity consistency and cancellation handling: 20
- Regression tests and edge cases: 15

- Code quality and maintainability: 5
- Required problem-solving documentation: 15 (approach.md: 5, challenges.md: 5, optimizations.md: 5)
- Pull request, dispatch notes and live explanation: 15

The three process documents are assessed for specific, honest, evidence-based technical reflection, not writing style.